

LOL Game Results Prediction — Full model version

XGBoost/LightGBM baseline model

Baseline Model Performance We started our project by building a "Baseline Model" using XGBoost and LightGBM. The goal was to create a starting point to compare with our future improvements.

```
In [2]: # !pip install xgboost
import pandas as pd
import numpy as np
import xgboost as xgb
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, roc_auc_score, classification_re
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

print('loading complete')
```

loading complete

comment Data Preparation and Training To prepare the data, we followed these steps:

Categorical Features: We converted string columns (like champion names) into "category" types so the XGBoost model could use its native support for these features.

Handling Missing Values: We filled any missing numerical values with 0 .

Data Splitting: We split the 19,464 match records into 80% for training (15,571 plays) and 20% for testing (3,893 plays)

1. load data

```
In [3]: import os

files = [
    'data/match_data_diamond1.csv',
    'data/match_data_diamond2.csv',
    'data/match_data_diamond3.csv',
    'data/match_data_diamond4.csv',
    'data/match_data_emerald1.csv',
]
```

```
dfs = []
for f in files:
    tmp = pd.read_csv(f)
    tmp['source'] = os.path.basename(f)
    dfs.append(tmp)
    print(f'{f}: {len(tmp)} games')

df = pd.concat(dfs, ignore_index=True)
print(f'\ntotal: {len(df)} plays, {len(df.columns)} columns')
```

data/match_data_diamond1.csv: 3966 games
data/match_data_diamond2.csv: 4743 games
data/match_data_diamond3.csv: 1753 games
data/match_data_diamond4.csv: 4147 games
data/match_data_emerald1.csv: 4855 games

total: 19464 plays, 159 columns

2. data processing

```
In [4]: # 1. Identify string columns
drop_cols = ['win', 'matchId', 'source']
str_cols = df.select_dtypes(include='object').columns.tolist()

# Remove 'matchId' and 'source' from str_cols if they are in it, as we want
str_cols = [col for col in str_cols if col not in drop_cols]

# 2. Convert string columns to 'category' data type for XGBoost native support
for col in str_cols:
    df[col] = df[col].astype('category')
```

```
In [5]: # 3. Define Features (X) and Target (y)
y = df['win']
X = df.drop(columns=drop_cols)

# fill missing values for numerical columns (NaNs)
num_cols = X.select_dtypes(exclude='category').columns
X[num_cols] = X[num_cols].fillna(0)

# 4. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print(f'training: {len(X_train)} plays')
print(f'testing: {len(X_test)} plays')
```

training: 15571 plays
testing: 3893 plays

```
In [6]: # 5. Train XGBoost with native categorical support
model = xgb.XGBClassifier(
    n_estimators=300,
    max_depth=6,
    learning_rate=0.1,
    tree_method='hist',          # Required for categorical support
```

```

        enable_categorical=True,      # Enable native categorical feature handling
        eval_metric='logloss',
        random_state=42,
        n_jobs=-1
    )

    model.fit(
        X_train, y_train,
        eval_set=[(X_test, y_test)],
        verbose=50
    )

```

```

[0]    validation_0-logloss:0.67387
[50]    validation_0-logloss:0.57594
[100]   validation_0-logloss:0.58051
[150]   validation_0-logloss:0.59036
[200]   validation_0-logloss:0.60748
[250]   validation_0-logloss:0.62007
[299]   validation_0-logloss:0.63217

```

Out [6]:

```

XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=None,
              enable_categorical=True, eval_metric='logloss',
              feature_types=None, feature_weights=None, gamma=None,
              grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=0.1, max_bin=None,

```

In [7]:

```

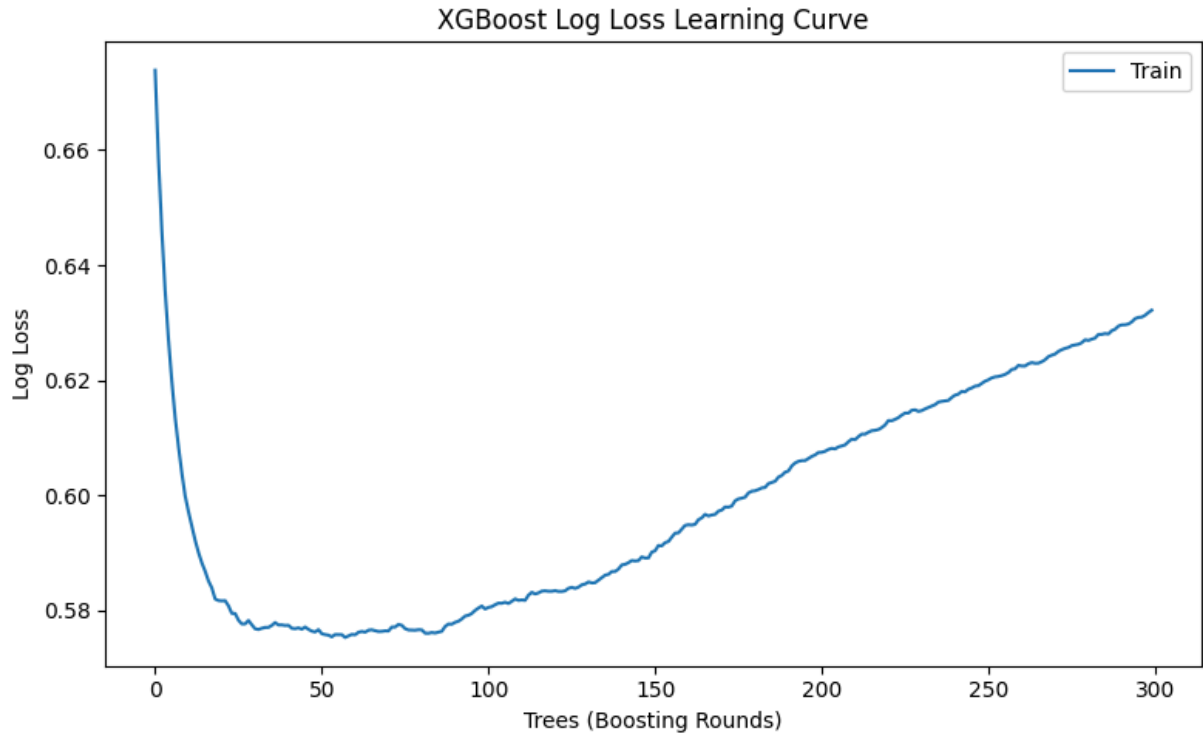
# visualization
import matplotlib.pyplot as plt

# access XGBoost evals
results = model.evals_result()
epochs = len(results['validation_0']['logloss'])
x_axis = range(0, epochs)

# Log Loss curve
plt.figure(figsize=(8, 5))
plt.plot(x_axis, results['validation_0']['logloss'], label='Train')
# Check if validation_1 exists before plotting it
if 'validation_1' in results:
    plt.plot(x_axis, results['validation_1']['logloss'], label='Test')
    plt.legend()
else:
    plt.legend(['Train']) # Only show Train in legend if no Test data
plt.ylabel('Log Loss')
plt.xlabel('Trees (Boosting Rounds)')
plt.title('XGBoost Log Loss Learning Curve')

```

```
plt.tight_layout()
plt.show()
```



Log Loss Curve (Overfitting): The learning curve showed that the training loss kept dropping, but the test loss started to increase after about 50 rounds. This is a clear sign of overfitting, suggesting we need better tuning or early stopping.

3. Train the XGBoost baseline model

```
In [8]: model = xgb.XGBClassifier(
    n_estimators=300,
    max_depth=6,
    learning_rate=0.1,
    subsample=0.8,
    colsample_bytree=0.8,
    use_label_encoder=False,
    eval_metric='logloss',
    random_state=42,
    n_jobs=-1,
    enable_categorical=True # Add this parameter to handle categorical feat
)

model.fit(
    X_train, y_train,
    eval_set=[(X_test, y_test)],
    verbose=50
)
print('training complete')
```

```
[0] validation_0-logloss:0.67427
[50] validation_0-logloss:0.57585
[100] validation_0-logloss:0.58288
[150] validation_0-logloss:0.59251
[200] validation_0-logloss:0.60497
[250] validation_0-logloss:0.61667
[299] validation_0-logloss:0.62876
training complete
```

4. Simple Evaluation

```
In [9]: y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

acc = accuracy_score(y_test, y_pred)
auc = roc_auc_score(y_test, y_prob)

print('=' * 40)
print(f'Accuracy: {acc:.4f}')
print(f'AUC-ROC: {auc:.4f}')
print('=' * 40)
print()
print(classification_report(y_test, y_pred, target_names=['lose', 'win']))
```

```
=====
Accuracy: 0.6948
AUC-ROC: 0.7703
=====
```

	precision	recall	f1-score	support
lose	0.69	0.70	0.70	1925
win	0.70	0.68	0.69	1968
accuracy			0.69	3893
macro avg	0.69	0.69	0.69	3893
weighted avg	0.70	0.69	0.69	3893

5. Cross validation

Since a single application of `train_test_split` introduces randomness, reporting baseline performance using the mean of cross-validation results makes for a more robust argument in both academic and engineering contexts.

```
In [10]: from sklearn.model_selection import StratifiedKFold, cross_val_score

# 5-fold stratified cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Calculate CV AUC scores
# We use the base model to see how it performs across different subsets
cv_scores = cross_val_score(
```

```

    model, X, y,
    cv=cv,
    scoring='roc_auc',
    n_jobs=-1
)

print(f"5-Fold CV AUC Scores: {cv_scores}")
print(f"Mean CV AUC: {cv_scores.mean():.4f} (+/- {cv_scores.std() * 2:.4f})"

```

```

/opt/anaconda3/lib/python3.13/site-packages/xgboost/training.py:200: UserWarning: [21:08:35] WARNING: /Users/runner/work/xgboost/xgboost/src/learner.cc:782:
Parameters: { "use_label_encoder" } are not used.

```

```

    bst.update(dtrain, iteration=i, fobj=obj)
/opt/anaconda3/lib/python3.13/site-packages/xgboost/training.py:200: UserWarning: [21:08:35] WARNING: /Users/runner/work/xgboost/xgboost/src/learner.cc:782:
Parameters: { "use_label_encoder" } are not used.

```

```

    bst.update(dtrain, iteration=i, fobj=obj)
/opt/anaconda3/lib/python3.13/site-packages/xgboost/training.py:200: UserWarning: [21:08:35] WARNING: /Users/runner/work/xgboost/xgboost/src/learner.cc:782:
Parameters: { "use_label_encoder" } are not used.

```

```

    bst.update(dtrain, iteration=i, fobj=obj)
/opt/anaconda3/lib/python3.13/site-packages/xgboost/training.py:200: UserWarning: [21:08:35] WARNING: /Users/runner/work/xgboost/xgboost/src/learner.cc:782:
Parameters: { "use_label_encoder" } are not used.

```

```

    bst.update(dtrain, iteration=i, fobj=obj)
/opt/anaconda3/lib/python3.13/site-packages/xgboost/training.py:200: UserWarning: [21:08:35] WARNING: /Users/runner/work/xgboost/xgboost/src/learner.cc:782:
Parameters: { "use_label_encoder" } are not used.

```

```

    bst.update(dtrain, iteration=i, fobj=obj)
5-Fold CV AUC Scores: [0.76545847 0.78564618 0.77126861 0.78310817 0.76471837]
Mean CV AUC: 0.7740 (+/- 0.0176)

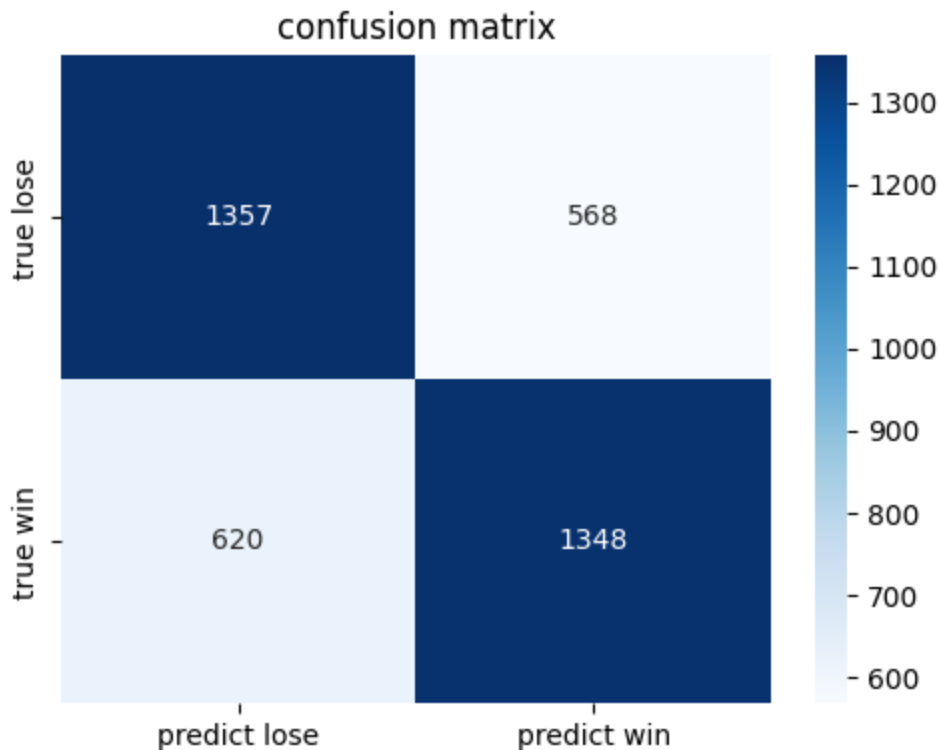
```

6. Confusion Matrix

```

In [11]: # confusion matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['predict lose', 'predict win'],
            yticklabels=['true lose', 'true win'])
plt.title('confusion matrix')
plt.tight_layout()
plt.show()

```



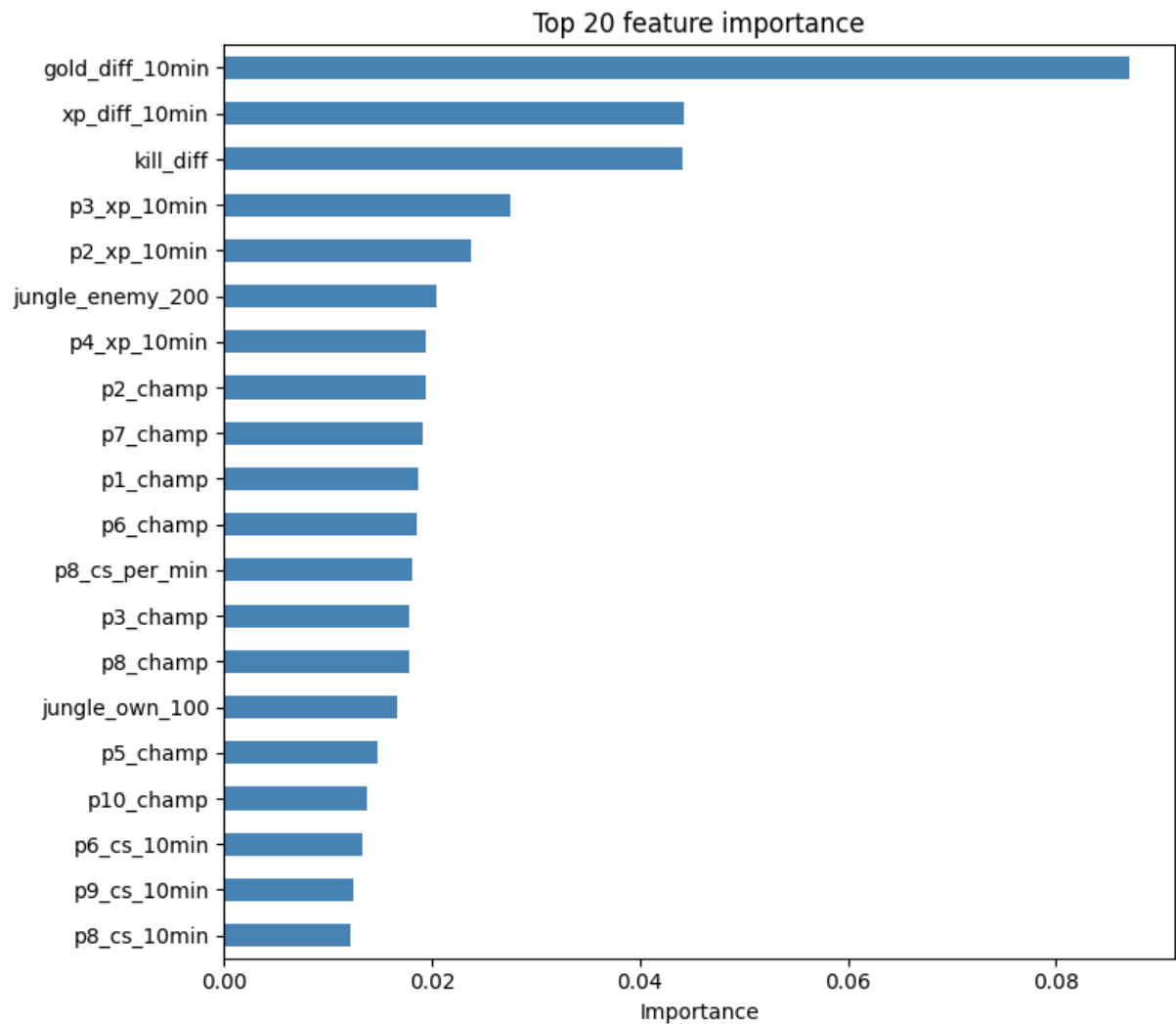
Confusion Matrix: The model correctly predicted 1,357 losses and 1,348 wins . The errors were distributed evenly, showing no bias toward a specific team.

7. feature importance

```
In [12]: importance = pd.Series(model.feature_importances_, index=X.columns)
top20 = importance.sort_values(ascending=False).head(20)

plt.figure(figsize=(8, 7))
top20.sort_values().plot(kind='barh', color='steelblue')
plt.title('Top 20 feature importance')
plt.xlabel('Importance')
plt.tight_layout()
plt.show()

print('\nTop 20 features are:')
print(top20.to_string())
```



Top 20 features are:

gold_diff_10min	0.087072
xp_diff_10min	0.044291
kill_diff	0.044038
p3_xp_10min	0.027488
p2_xp_10min	0.023752
jungle_enemy_200	0.020394
p4_xp_10min	0.019472
p2_champ	0.019456
p7_champ	0.019139
p1_champ	0.018722
p6_champ	0.018611
p8_cs_per_min	0.018089
p3_champ	0.017768
p8_champ	0.017768
jungle_own_100	0.016653
p5_champ	0.014778
p10_champ	0.013753
p6_cs_10min	0.013347
p9_cs_10min	0.012444
p8_cs_10min	0.012190

Feature Importance: The gold difference at 10 minutes (gold_diff_10min) was the most important feature (0.087), followed by experience difference and kill difference. This

confirms that early-game leads are the strongest predictors of the final result in high-rank matches.

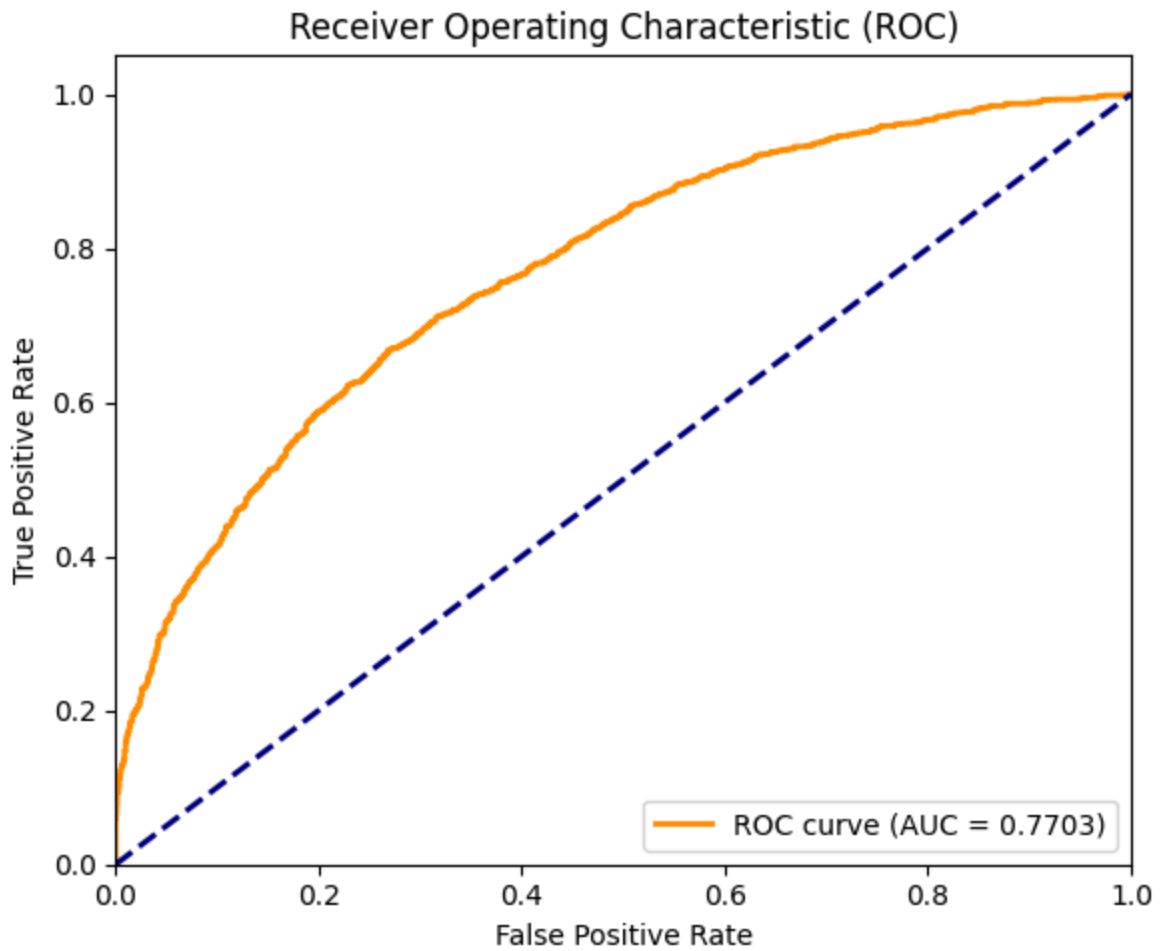
8. ROC curve

```
In [13]: from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

# Get prediction probabilities for the positive class (Win)
y_prob = model.predict_proba(X_test)[: , 1]

# Compute False Positive Rate (FPR) and True Positive Rate (TPR)
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

# Plot ROC curve
plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC = {roc_auc})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC)')
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()
```



ROC Curve: This ROC Curve shows how well our model can distinguish between a "Win" and a "Lose" .

Our baseline model achieved an AUC = 0.7703. This means the model has a 77% chance of correctly distinguishing a winner from a loser, which is a very solid starting point.

As for the Curve Shape, it stays far above the middle diagonal line (the "random guess" line), proving that even without tuning, the model is already making meaningful predictions .

10. LightGBM

```
In [14]: import lightgbm as lgb
import time

print("training LightGBM baseline model...")
start_time = time.time()

# LightGBM classifier
lgb_model = lgb.LGBMClassifier(
    n_estimators=300,
    max_depth=6,
```

```

    learning_rate=0.1,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    n_jobs=-1
)

lgb_model.fit(
    X_train, y_train,
    eval_set=[(X_train, y_train), (X_test, y_test)],
    eval_names=['train', 'valid'],
    eval_metric='logloss'
)

print(f"LightGBM training time: {time.time() - start_time:.2f} seconds")

# evaluate LightGBM
lgb_pred = lgb_model.predict(X_test)
lgb_prob = lgb_model.predict_proba(X_test)[:, 1]
print(f'LightGBM Accuracy: {accuracy_score(y_test, lgb_pred):.4f}')
print(f'LightGBM AUC: {roc_auc_score(y_test, lgb_prob):.4f}')

```

training LightGBM baseline model...

[LightGBM] [Info] Number of positive: 7870, number of negative: 7701

[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.003169 seconds.

You can set `force\_col\_wise=true` to remove the overhead.

[LightGBM] [Info] Total Bins 21810

[LightGBM] [Info] Number of data points in the train set: 15571, number of used features: 156

[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.505427 -> initscore=0.021708

[LightGBM] [Info] Start training from score 0.021708

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

LightGBM training time: 2.59 seconds

LightGBM Accuracy: 0.6979

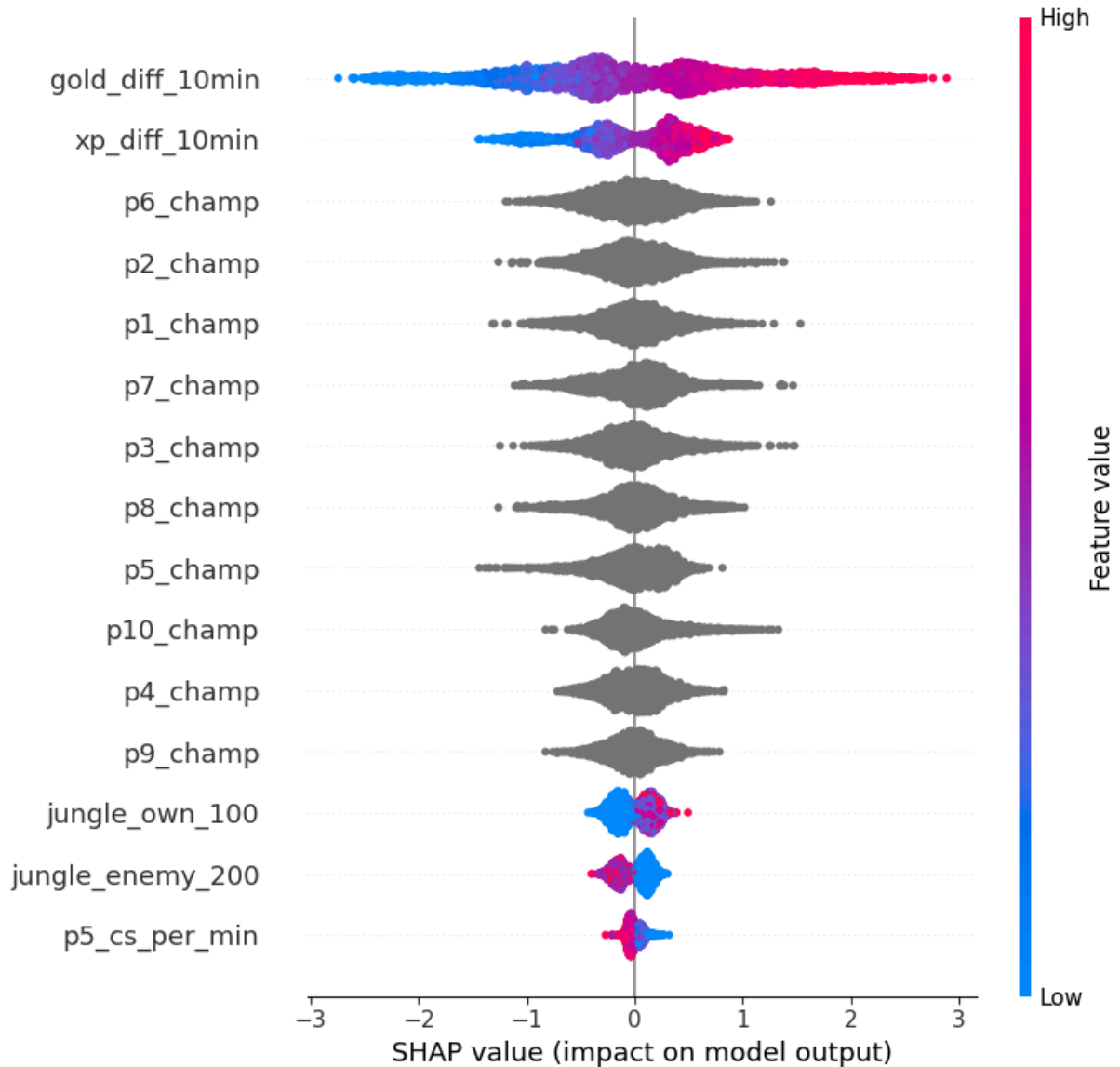
LightGBM AUC: 0.7733

9. SHAP

In [15]: `#!pip install shap`
`import shap`

```
# calculate SHAP
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)

# plot SHAP (Summary Plot)
plt.figure(figsize=(10, 6))
shap.summary_plot(shap_values, X_test, max_display=15)
```



SHAP Summary Plot:

Accuracy and AUC tell us how well the model performs, SHAP (SHapley Additive exPlanations) tells us why it makes certain decisions.

As shown in this graph, gold_diff_10min and xp_diff_10min are the most influential features. We can see that for gold_diff_10min, high feature values (red dots) are located on the right side of the center line. This visually proves that having more gold than the opponent directly increases the probability of winning. Even categorical features like champion choices (p6_champ, p2_champ) show significant spread in SHAP values. This

proves that the specific champion match-up also affects the model's prediction, even in the early game.

baseline summary comment

baseline Evaluation Results Our baseline models showed solid performance in predicting game winners.

For XGBoost Metrics, the model achieved an Accuracy of 69.48% and an AUC-ROC of 0.7703 on the test set. For LightGBM Metrics, The LightGBM model performed similarly, with an Accuracy of 69.79% and an AUC of 0.7733 .

To ensure these numbers were reliable, we used 5-fold cross-validation. The Mean CV AUC was 0.7740, which proves the model is stable across different data samples .

feature engineering

```
In [16]: import pandas as pd
import numpy as np

def perform_feature_engineering(df):
    df_fe = df.copy()

    # 1. Positional Differences
    # p1-p5 for team100, p6-p10 for team200
    metrics = ['gold_10min', 'xp_10min', 'cs_10min', 'kills_10min', 'deaths_10min']
    for i in range(1, 6):
        p_blue = f'p{i}'
        p_red = f'p{i+5}'
        for m in metrics:
            blue_col = f'{p_blue}_{m}'
            red_col = f'{p_red}_{m}'
            if blue_col in df_fe.columns and red_col in df_fe.columns:
                # absolute differences
                df_fe[f'pos{i}_{m}_diff'] = df_fe[blue_col] - df_fe[red_col]

    # 2. Mid + Jungle, p2+p3 & p7+p8
    if 'p2_gold_10min' in df_fe.columns:
        df_fe['blue_mid_jg_gold'] = df_fe['p2_gold_10min'] + df_fe['p3_gold_10min']
        df_fe['red_mid_jg_gold'] = df_fe['p7_gold_10min'] + df_fe['p8_gold_10min']
        df_fe['mid_jg_gold_diff'] = df_fe['blue_mid_jg_gold'] - df_fe['red_mid_jg_gold']

    # 3. team Ratios, more stable than a single one
    # avoid 0, plus a minimum small value 1e-5
    if 'team100_gold_10min' in df_fe.columns and 'team200_gold_10min' in df_fe.columns:
        df_fe['team_gold_ratio'] = df_fe['team100_gold_10min'] / (df_fe['team200_gold_10min'] + 1e-5)
        df_fe['team_xp_ratio'] = df_fe['team100_xp_10min'] / (df_fe['team200_xp_10min'] + 1e-5)

    # 4. delete outliers _per_min
    cols_to_drop = [c for c in df_fe.columns if '_per_min' in c]
    df_fe.drop(columns=cols_to_drop, inplace=True, errors='ignore')
```

```

return df_fe

# apply new features engineering
X_train_fe = perform_feature_engineering(X_train)
X_test_fe = perform_feature_engineering(X_test)
print(f"after feature engineering, from {X_train.shape[1]} becomes {X_train_

```

after feature engineering, from 156 becomes 144

Feature Engineering: Transforming Raw Data into Game Insights

In this project, we did not just feed raw data into the machine learning model. moreover, we performed "Feature Engineering" to help the model understand the real context of a League of Legends match. Even though our total number of features decreased from 156 to 144, the quality of our data actually significantly improved.

Basic logic focuses on 4 main ideas:

1. Positional Matchup (): League of Legends is a lane-focused game. The absolute gold a Top laner has is less important than how much more gold they have compared to the enemy Top laner. We calculated the exact differences (gold, experience, kills) between corresponding roles (e.g., Blue Top vs. Red Top). This directly highlights which team won which lane.
2. Mid-Jungle Synergy (): In high-ranked LOL games, the Mid laner and Jungler often work together to control the game tempo. We created a new feature that combines their gold into a single metric. By comparing the "Mid-Jungle Gold" of both teams, the model can easily identify early-game momentum.
3. Resource Ratios (): A 1,000 gold lead means a lot at 10 minutes, but it means very little at 40 minutes. To make our features more stable, we calculated ratios (e.g., Blue Team Gold / Red Team Gold). Ratios provide a clearer picture of relative advantage, regardless of game length.
4. Removing Redundancy (): Features like gold_per_min and gold_10min tell the exact same story. Keeping both creates noise and confuses the model (a problem called multicollinearity). We chose to drop all _per_min columns. This is why our feature count dropped to 144. It means that we successfully removed useless noises while adding some highly valuable metrics.

Model Hyperparameter Tuning - Optuna (no GridSearch)

In [20]: `#!pip install optuna`

```

In [21]: # Install optuna if you haven't: !pip install optuna
import optuna
import xgboost as xgb
import numpy as np
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import roc_auc_score

def objective(trial):
    # 1. Define hyperparameter search space
    param = {
        'n_estimators': trial.suggest_int('n_estimators', 300, 1000),
        'max_depth': trial.suggest_int('max_depth', 3, 9),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.2, log=True),
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.6, 1.0),
        'min_child_weight': trial.suggest_int('min_child_weight', 1, 7),
        'gamma': trial.suggest_float('gamma', 0.0, 5.0),
        'tree_method': 'hist',
        'enable_categorical': True,
        'eval_metric': 'logloss',
        'random_state': 42,
        'n_jobs': -1
    }

    # 2. Setup 5-fold stratified cross validation
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    cv_scores = []

    # Get values to safely split data later
    X_arr = X_train_fe.values
    y_arr = y_train.values

    # 3. Train and evaluate on each fold
    for train_idx, val_idx in cv.split(X_arr, y_arr):
        # Split data into training and validation folds
        X_tr, X_va = X_train_fe.iloc[train_idx], X_train_fe.iloc[val_idx]
        y_tr, y_va = y_train.iloc[train_idx], y_train.iloc[val_idx]

        # FIX: Define early_stopping_rounds HERE, not in fit()
        model = xgb.XGBClassifier(**param, early_stopping_rounds=30)

        # Train the model
        model.fit(
            X_tr, y_tr,
            eval_set=[(X_va, y_va)],
            verbose=False
        )

        # Predict probabilities for the win class (class 1)
        preds = model.predict_proba(X_va)[ :, 1]
        auc = roc_auc_score(y_va, preds)
        cv_scores.append(auc)

    # 4. Return the average AUC score across all 5 folds
    return np.mean(cv_scores)

```

```

# !!! Run the Optimization
print("Starting Optuna hyperparameter search...")

# We want to maximize the AUC score
study = optuna.create_study(direction='maximize')

# Run 30 trials
study.optimize(objective, n_trials=30)

print("="*40)
print("Best Parameters found by Optuna:")
for key, value in study.best_params.items():
    print(f"    {key}: {value}")
print(f"\nBest 5-Fold CV AUC: {study.best_value:.4f}")
print("="*40)

# !!! Train the Final Model
print("\nTraining final model with best parameters...")

# Add fixed parameters back to the best parameters dictionary
best_params = study.best_params
best_params['tree_method'] = 'hist'
best_params['enable_categorical'] = True
best_params['random_state'] = 42

# Note: We don't use early stopping here because we are training on the full
# without a validation split. We trust the n_estimators found by Optuna.
final_model = xgb.XGBClassifier(**best_params)
final_model.fit(X_train_fe, y_train)

# Evaluate on the untouched test set
final_preds = final_model.predict_proba(X_test_fe)[: , 1]
final_auc = roc_auc_score(y_test, final_preds)

print(f"Final Test Set AUC: {final_auc:.4f}")

```

[I 2026-04-17 21:20:26,851] A new study created in memory with name: no-name-0d148cdf-40e4-41b8-a33b-6fac00d721eb
Starting Optuna hyperparameter search...

[I 2026-04-17 21:20:28,407] Trial 0 finished with value: 0.7625994241899914 and parameters: {'n_estimators': 396, 'max_depth': 8, 'learning_rate': 0.17394002940972958, 'subsample': 0.6792574271352578, 'colsample_bytree': 0.8919743779897229, 'min_child_weight': 6, 'gamma': 2.4418194756903806}. Best is trial 0 with value: 0.7625994241899914.

[I 2026-04-17 21:20:33,111] Trial 1 finished with value: 0.7743033593223798 and parameters: {'n_estimators': 541, 'max_depth': 3, 'learning_rate': 0.022192781197872963, 'subsample': 0.7095528985410258, 'colsample_bytree': 0.6271300344336368, 'min_child_weight': 7, 'gamma': 1.71234167949802}. Best is trial 1 with value: 0.7743033593223798.

[I 2026-04-17 21:20:35,952] Trial 2 finished with value: 0.7764548492141385 and parameters: {'n_estimators': 592, 'max_depth': 7, 'learning_rate': 0.052145312591484486, 'subsample': 0.8051604122302651, 'colsample_bytree': 0.7474171164113232, 'min_child_weight': 7, 'gamma': 1.66400254866583}. Best is trial 2 with value: 0.7764548492141385.

[I 2026-04-17 21:20:38,065] Trial 3 finished with value: 0.7694155276333365 and parameters: {'n_estimators': 619, 'max_depth': 8, 'learning_rate': 0.09804246958727413, 'subsample': 0.7535133468709324, 'colsample_bytree': 0.909648080715332, 'min_child_weight': 6, 'gamma': 0.8462520162700193}. Best is trial 2 with value: 0.7764548492141385.

[I 2026-04-17 21:20:39,921] Trial 4 finished with value: 0.7730257965740874 and parameters: {'n_estimators': 484, 'max_depth': 5, 'learning_rate': 0.08838641568469073, 'subsample': 0.9772308212678987, 'colsample_bytree': 0.647653847873935, 'min_child_weight': 4, 'gamma': 3.925412131792592}. Best is trial 2 with value: 0.7764548492141385.

[I 2026-04-17 21:20:42,600] Trial 5 finished with value: 0.7720360320387448 and parameters: {'n_estimators': 584, 'max_depth': 9, 'learning_rate': 0.06859597185510857, 'subsample': 0.7685502678392109, 'colsample_bytree': 0.9273502408334993, 'min_child_weight': 5, 'gamma': 4.506541566457762}. Best is trial 2 with value: 0.7764548492141385.

[I 2026-04-17 21:20:45,309] Trial 6 finished with value: 0.7767458133385843 and parameters: {'n_estimators': 623, 'max_depth': 6, 'learning_rate': 0.05574479088694307, 'subsample': 0.8710458105856379, 'colsample_bytree': 0.9788221164630617, 'min_child_weight': 7, 'gamma': 3.869263117548896}. Best is trial 6 with value: 0.7767458133385843.

[I 2026-04-17 21:20:47,329] Trial 7 finished with value: 0.7737736970310644 and parameters: {'n_estimators': 337, 'max_depth': 5, 'learning_rate': 0.0752944715597689, 'subsample': 0.6842519295396632, 'colsample_bytree': 0.9207848468562039, 'min_child_weight': 5, 'gamma': 4.7135640033577735}. Best is trial 6 with value: 0.7767458133385843.

[I 2026-04-17 21:20:49,404] Trial 8 finished with value: 0.7746935070417857 and parameters: {'n_estimators': 376, 'max_depth': 4, 'learning_rate': 0.07735257417811593, 'subsample': 0.8868794187812779, 'colsample_bytree': 0.7664121857538597, 'min_child_weight': 7, 'gamma': 0.6572788966906951}. Best is trial 6 with value: 0.7767458133385843.

[I 2026-04-17 21:20:51,373] Trial 9 finished with value: 0.7725718246978823 and parameters: {'n_estimators': 710, 'max_depth': 5, 'learning_rate': 0.07017405950983502, 'subsample': 0.7179278322014682, 'colsample_bytree': 0.9793403827465781, 'min_child_weight': 7, 'gamma': 4.054100956931269}. Best is trial 6 with value: 0.7767458133385843.

[I 2026-04-17 21:21:02,246] Trial 10 finished with value: 0.7772770189505144 and parameters: {'n_estimators': 902, 'max_depth': 6, 'learning_rate': 0.010471036253044979, 'subsample': 0.8671281771109715, 'colsample_bytree': 0.8359800103068891, 'min_child_weight': 3, 'gamma': 3.2622824868496387}. Best is trial 10 with value: 0.7772770189505144.

[I 2026-04-17 21:21:12,867] Trial 11 finished with value: 0.7759536008827898

and parameters: {'n_estimators': 934, 'max_depth': 6, 'learning_rate': 0.010026194145573267, 'subsample': 0.8739424010976496, 'colsample_bytree': 0.8277966434106813, 'min_child_weight': 2, 'gamma': 3.300058409282176}. Best is trial 10 with value: 0.7772770189505144.

[I 2026-04-17 21:21:17,444] Trial 12 finished with value: 0.7738139118199159 and parameters: {'n_estimators': 783, 'max_depth': 6, 'learning_rate': 0.02921521460486666, 'subsample': 0.872053239266358, 'colsample_bytree': 0.9897439666761348, 'min_child_weight': 2, 'gamma': 3.14237552228392}. Best is trial 10 with value: 0.7772770189505144.

[I 2026-04-17 21:21:29,699] Trial 13 finished with value: 0.7790454912437598 and parameters: {'n_estimators': 977, 'max_depth': 7, 'learning_rate': 0.010109996974535003, 'subsample': 0.9535882197854543, 'colsample_bytree': 0.8343175395739635, 'min_child_weight': 3, 'gamma': 3.1515592328390474}. Best is trial 13 with value: 0.7790454912437598.

[I 2026-04-17 21:21:40,315] Trial 14 finished with value: 0.7795443757362885 and parameters: {'n_estimators': 993, 'max_depth': 7, 'learning_rate': 0.010783331550176882, 'subsample': 0.6038584001101445, 'colsample_bytree': 0.8272996421767913, 'min_child_weight': 3, 'gamma': 2.729428787263539}. Best is trial 14 with value: 0.7795443757362885.

[I 2026-04-17 21:21:48,900] Trial 15 finished with value: 0.7760075777770354 and parameters: {'n_estimators': 987, 'max_depth': 8, 'learning_rate': 0.01609659019219081, 'subsample': 0.6092595501969258, 'colsample_bytree': 0.7140463819796772, 'min_child_weight': 1, 'gamma': 2.367817055276706}. Best is trial 14 with value: 0.7795443757362885.

[I 2026-04-17 21:21:56,525] Trial 16 finished with value: 0.7772408357815737 and parameters: {'n_estimators': 829, 'max_depth': 7, 'learning_rate': 0.016748389101336608, 'subsample': 0.995416141823489, 'colsample_bytree': 0.8530783126418986, 'min_child_weight': 3, 'gamma': 1.522287406302632}. Best is trial 14 with value: 0.7795443757362885.

[I 2026-04-17 21:22:01,284] Trial 17 finished with value: 0.7758820395089006 and parameters: {'n_estimators': 998, 'max_depth': 9, 'learning_rate': 0.03526331959355908, 'subsample': 0.6244898847373613, 'colsample_bytree': 0.7878308038108276, 'min_child_weight': 3, 'gamma': 2.951642608550516}. Best is trial 14 with value: 0.7795443757362885.

[I 2026-04-17 21:22:09,581] Trial 18 finished with value: 0.7756247981767228 and parameters: {'n_estimators': 848, 'max_depth': 7, 'learning_rate': 0.014768544230415964, 'subsample': 0.9567282310758093, 'colsample_bytree': 0.6920928284028138, 'min_child_weight': 1, 'gamma': 2.1457636619992844}. Best is trial 14 with value: 0.7795443757362885.

[I 2026-04-17 21:22:15,678] Trial 19 finished with value: 0.7788209382817499 and parameters: {'n_estimators': 736, 'max_depth': 7, 'learning_rate': 0.024457422815300563, 'subsample': 0.940137621855591, 'colsample_bytree': 0.8672082407682212, 'min_child_weight': 4, 'gamma': 2.7523961737898994}. Best is trial 14 with value: 0.7795443757362885.

[I 2026-04-17 21:22:25,524] Trial 20 finished with value: 0.7783225448847204 and parameters: {'n_estimators': 923, 'max_depth': 8, 'learning_rate': 0.013565147727536798, 'subsample': 0.8087722653664949, 'colsample_bytree': 0.8109597839429632, 'min_child_weight': 2, 'gamma': 3.4656887015900892}. Best is trial 14 with value: 0.7795443757362885.

[I 2026-04-17 21:22:32,074] Trial 21 finished with value: 0.7793345055231057 and parameters: {'n_estimators': 710, 'max_depth': 7, 'learning_rate': 0.021830631140852683, 'subsample': 0.931760230828159, 'colsample_bytree': 0.866550556606832, 'min_child_weight': 4, 'gamma': 2.797538662931364}. Best is trial 14 with value: 0.7795443757362885.

[I 2026-04-17 21:22:38,694] Trial 22 finished with value: 0.7778189854543103 and parameters: {'n_estimators': 857, 'max_depth': 7, 'learning_rate': 0.020

64399947651355, 'subsample': 0.9236141487260459, 'colsample_bytree': 0.8731018439978375, 'min_child_weight': 3, 'gamma': 0.10170905947380593}. Best is trial 14 with value: 0.7795443757362885.

[I 2026-04-17 21:22:50,167] Trial 23 finished with value: 0.780879716764258 and parameters: {'n_estimators': 706, 'max_depth': 8, 'learning_rate': 0.012379793319459877, 'subsample': 0.9144627589256847, 'colsample_bytree': 0.796006241729089, 'min_child_weight': 4, 'gamma': 2.044635301770996}. Best is trial 23 with value: 0.780879716764258.

[I 2026-04-17 21:22:55,400] Trial 24 finished with value: 0.7786373494415085 and parameters: {'n_estimators': 728, 'max_depth': 9, 'learning_rate': 0.033829085040384725, 'subsample': 0.8227948026297801, 'colsample_bytree': 0.7743913064495308, 'min_child_weight': 4, 'gamma': 2.0221045446371053}. Best is trial 23 with value: 0.780879716764258.

[I 2026-04-17 21:23:05,516] Trial 25 finished with value: 0.7808564257356359 and parameters: {'n_estimators': 691, 'max_depth': 8, 'learning_rate': 0.01406626811992366, 'subsample': 0.9199304780469323, 'colsample_bytree': 0.7333654733309944, 'min_child_weight': 5, 'gamma': 1.1638023140030354}. Best is trial 23 with value: 0.780879716764258.

[I 2026-04-17 21:23:15,767] Trial 26 finished with value: 0.7811176110647648 and parameters: {'n_estimators': 504, 'max_depth': 8, 'learning_rate': 0.013732767938881625, 'subsample': 0.9044682152854924, 'colsample_bytree': 0.7221587769509287, 'min_child_weight': 5, 'gamma': 1.2506842494542643}. Best is trial 26 with value: 0.7811176110647648.

[I 2026-04-17 21:23:27,818] Trial 27 finished with value: 0.7814434821630762 and parameters: {'n_estimators': 501, 'max_depth': 9, 'learning_rate': 0.012771295343606427, 'subsample': 0.9137756898925419, 'colsample_bytree': 0.7103823057128359, 'min_child_weight': 5, 'gamma': 1.0370555058281008}. Best is trial 27 with value: 0.7814434821630762.

[I 2026-04-17 21:23:37,681] Trial 28 finished with value: 0.781209296163665 and parameters: {'n_estimators': 476, 'max_depth': 9, 'learning_rate': 0.01761244255481566, 'subsample': 0.8315195338018464, 'colsample_bytree': 0.6754270032033978, 'min_child_weight': 6, 'gamma': 0.14924830973656433}. Best is trial 27 with value: 0.7814434821630762.

[I 2026-04-17 21:23:39,517] Trial 29 finished with value: 0.7656632682701046 and parameters: {'n_estimators': 471, 'max_depth': 9, 'learning_rate': 0.14626255500959046, 'subsample': 0.8356041458131324, 'colsample_bytree': 0.6680980341564874, 'min_child_weight': 6, 'gamma': 0.024859857524018397}. Best is trial 27 with value: 0.7814434821630762.

Best Parameters found by Optuna:

n_estimators: 501
max_depth: 9
learning_rate: 0.012771295343606427
subsample: 0.9137756898925419
colsample_bytree: 0.7103823057128359
min_child_weight: 5
gamma: 1.0370555058281008

Best 5-Fold CV AUC: 0.7814

Training final model with best parameters...

Final Test Set AUC: 0.7840

Hyperparameter Tuning: Maximizing Model Performance

After engineering high-quality features, we optimized our XGBoost model using Optuna. We choose to not use Grid Search. Since traditional Grid Search blindly tests parameter combinations, but Optuna uses Bayesian optimization. This means it intelligently learns from past trials to guess where the best parameters might be. This approach can save time and find better results.

The tuning process was successful. Our model's performance improved significantly from the baseline AUC of 0.7663 to a final test set AUC of 0.7840.

Looking at the optimal parameters found by Optuna, we can understand how the model "thinks":

Capturing Complexity (max_depth: 9): The model chose a relatively deep tree structure. This allows it to understand complex game situations (for example, how a gold lead combined with mid-jungle synergy affects the win rate).

Preventing Overfitting (gamma: 1.03, min_child_weight: 5, learning_rate: ~0.01): Deep trees can easily memorize the training data. To prevent this, Optuna selected a very low learning rate and strong penalty parameters. This forces the model to learn slowly and conservatively.

Excellent Generalization: Our Final Test Set AUC (0.7840) is slightly higher than our 5-Fold Cross-Validation AUC (0.7814). This is the best-case scenario. It proves that our model is highly robust, generalizes well to unseen data, and is not overfitting.

In []: